

오픈소스SW의 이해

CHAPTER 4

SEMESTER | 2026-2

박성미

The background image shows a wide-angle view of a university campus. In the center is a large, modern building with a glass facade and a flat roof. To the left of the building is a large, open green space with trees. In the foreground, there is a paved area with some parked cars and a few people walking. The entire image is covered with a semi-transparent blue overlay.

클래스

한림대학교

- 객체 지향 프로그래밍(Object Oriented Programming, OOP)
 - 내가 아닌 남이 만든 코드를 재사용하고 싶을 때 사용하는 대표적인 방법
 - 함수처럼 어떤 기능을 함수 코드로 묶어 두는 것이 아니라,
어떤 기능을 수행하는 하나의 단일 프로그램을 객체라고 하는 코드로 만들어
다른 프로그래머가 재사용할 수 있도록 함

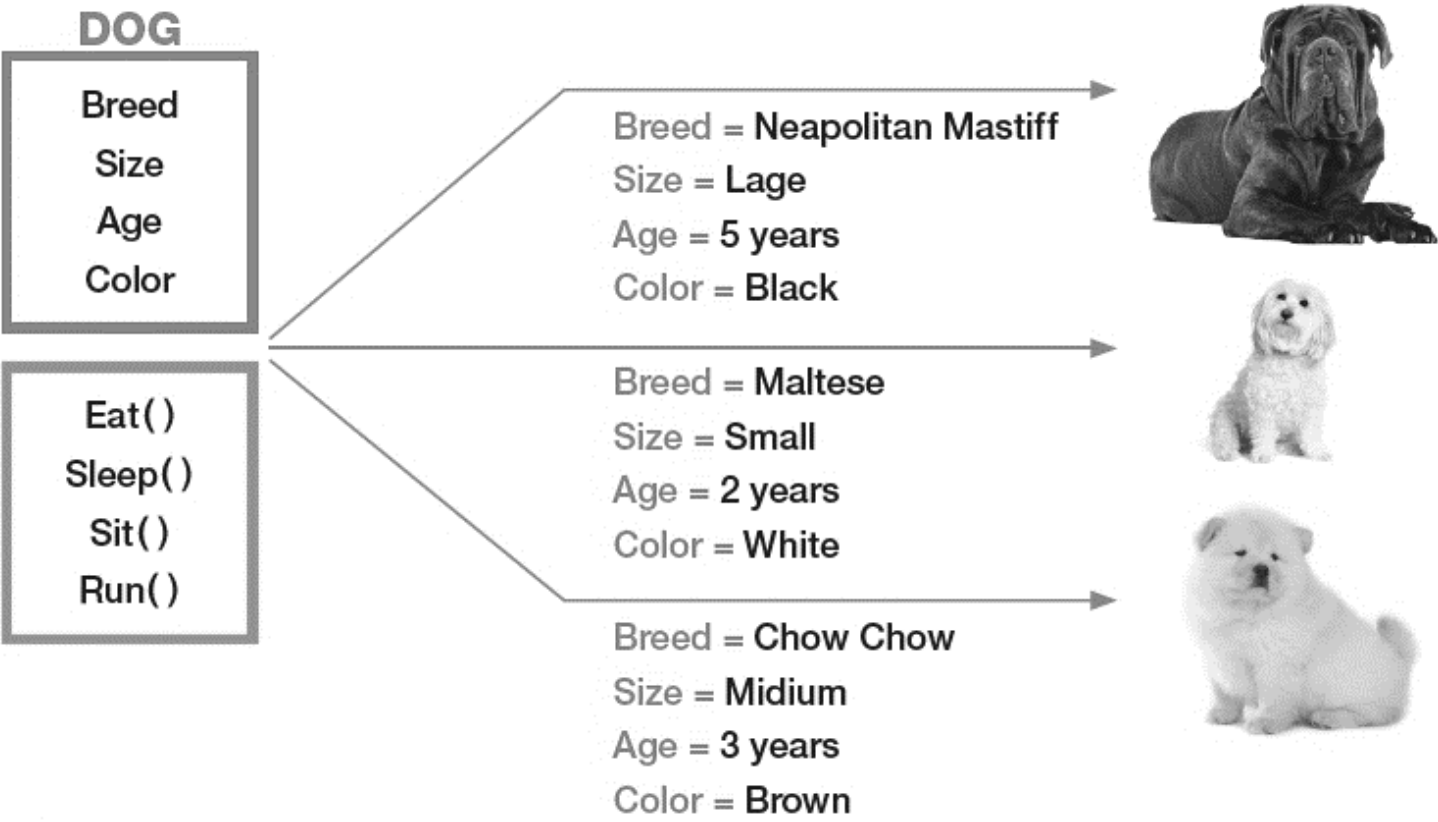
- 객체(object): 실생활에 존재하는 실제적인 물건 또는 개념
- 객체 지향 프로그래밍: 객체의 개념을 활용하여 프로그램으로 표현하는 기법

개념	설명	예시
객체(object)	실생활에 존재하는 실제적인 물건 또는 개념	심판, 선수, 팀
속성(attribute)	객체가 가지고 있는 변수	선수의 이름, 포지션, 소속팀
행동(action)	객체가 실제로 작동시키는 함수, 메서드	공을 차다, 패스하다

객체는 하나의 프로그램에서 여러 개가 사용될 수도 있으므로 객체들을 위한 설계도를 만들어야 함

👉 클래스(class): 객체가 가져야 할 기본 정보를 담은 코드, 일종의 설계도 코드

- 'DOG' 클래스
 - 실제 사용하는 종류별로 Dog 인스턴스를 만들 수 있음.



붕어빵틀
(클래스)



붕어빵
(인스턴스)

클래스

■ 파이썬에 클래스를 선언하기 위한 기본 코드 템플릿

class SoccerPlayer(object):

↑ ↑ ↑

클래스 예약어 클래스 이름 상속받는 객체명

```
class Dog: # 가장 기본적인 형태
    pass
```

```
class SmartPhone: # 두 단어 이상 결합 시
    pass
```

예약어인 class를 코드의 맨 앞에 입력하고, 만들고자 하는 클래스 이름을 작성

그 다음으로 상속받아야 하는 다른 클래스의 이름을 괄호 안에 넣음

작명 기법	설명
snake_case	띄어쓰기 부분에 '_'를 추가하여 변수의 이름을 지정한다. 파이썬 함수나 변수명에 사용된다.
CamelCase	띄어쓰기 부분에 대문자를 사용하여 변수의 이름을 지정한다. 낙타의 혹처럼 생겼다 하여 Camel이라고 부르고, 주로 파이썬 클래스명에 사용된다.

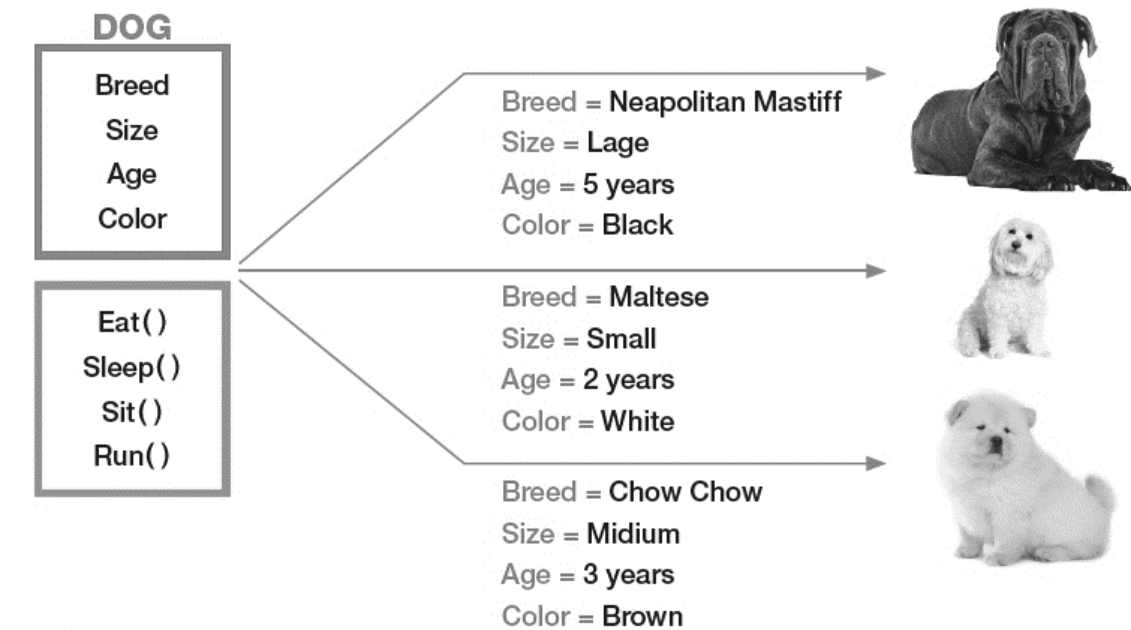
클래스

■ 속성의 선언

- Dog 클래스 구성
- 먼저 클래스의 속성을 추가하는 선언
 - 속성에 대한 정보를 선언하기 위해서는 **`__init__()`**라는 예약 함수 사용
 - 파이썬 클래스의 대표적 예약 함수: `__init__()`, `__str__`, `__add__` 등

```
class Dog:
    def __init__(self, breed, size, age, color):
        self.breed = breed
        self.size = size
        self.age = age
        self.color = color
        self.hunger_level = 100

#객체 생성시 __init__ 자동 호출하게 됨
my_dog = Dog("Maltese", "Small", "2years", "Brown")
```



`__init__`: 객체가 생성될 때 자동으로 호출되는 메서드 (생성자)

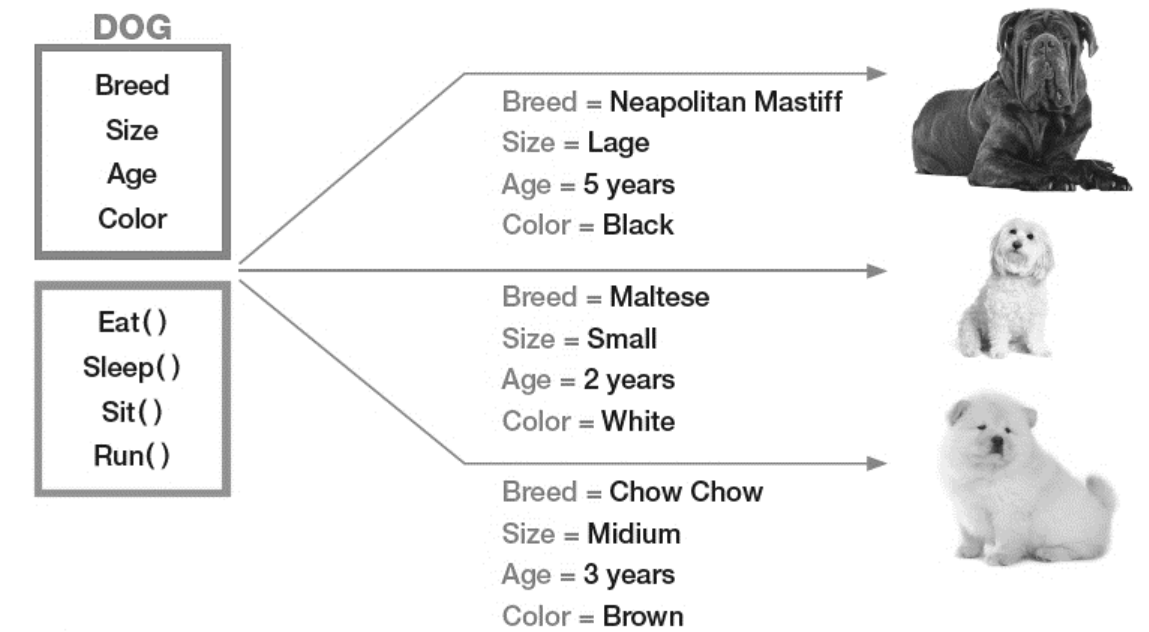
`self`: 객체 자신을 가리키는 첫 번째 매개변수. 메서드 정의 시 반드시 첫 번째 인자로 `self`를 포함해야 함

클래스

■ 함수의 선언

- 함수는 이 클래스가 할 수 있는 **다양한 동작을 정의할 수 있음**
- 강아지가 음식을 먹는 행동을 함수 코드로 표현하기

```
class Dog:
    ...
    def eat(self, food_amout):
        self.hunger_level -= food_amount
        print(f"{self.breed}가 사료 {food}그릇을 먹었다. 현재 배고픔 정도: {self.hunger_level}")
    ...
my_dog = Dog("Maltese", "Small", "2years", "Brown")
my_dog.eat(10)
```



- 클래스 내에서의 함수도 함수의 이름을 쓰고 매개변수를 사용함

반드시 self를 매개변수에 넣을 것!

- 클래스를 만들어봅시다.

1. 강아지 클래스 정의하기

Dog 클래스는 다음 속성을 가짐:

name: 강아지 이름

breed: 견종

age: 나이

hunger_level: 배고픔 정도, 초기값 100

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name
        self.breed = breed
        self.age = age
        self.hunger_level = 100
        self.energy_level = 100
```

클래스

- 클래스를 만들어봅시다.

2. 메서드 완성하기

- eat(amount)
 - 먹은 양만큼 hunger_level 감소
 - 배고픔 정도가 0보다 작아지지 않도록 함

```
def eat(self, amount):  
    self.hunger_level -= amount  
  
    if self.hunger_level < 0:  
        self.hunger_level = 0  
  
    print(f"{self.name}가 사료를 {amount}만큼 먹었습니다.")
```

클래스

- 클래스를 만들어봅시다.

3. 호출해보기

- 이름이 “초코”인 2살 말티즈 객체 생성
- 사료를 30만큼 먹임

```
my_dog = Dog("초코", "Maltese", 2)
```

```
my_dog.eat(30)
```

클래스

■ 클래스 사용해보기 1

CLASSUM

클래스

- 활동 레벨(energy_level)을 추가함

메서드 완성하기

- play(minutes)
 - 놀이 시간의 2배만큼 **energy_level**을 감소
 - 놀이 시간만큼 **hunger_level**을 증가
 - 에너지가 부족하면 "에너지가 부족합니다."를 출력
- sleep(hours)
 - 잠든 시간의 20배만큼 energy_level을 증가
 - 에너지 정도가 100보다 커지지 않도록 함!!!
- show_status()
 - 강아지의 이름, 견종, 나이, 배고픔 정도, 에너지 정도를 출력

클래스

■ 클래스 사용해보기 1

3. 호출해보기

- 이름이 흰둥이인 5살 진돗개 객체를 생성
- 사료를 50만큼 먹임
- 20분 동안 놀게 함
- 2시간 동안 재움
- 현재 상태를 출력

CLASSUM

클래스

클래스 사용해보기 1

CLASSUM

정답

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name
        self.breed = breed
        self.age = age
        self.hunger_level = 100
        self.energy_level = 100

    def eat(self, amount):
        if amount <= 0:
            print("먹이의 양은 1 이상이어야 합니다.")
            return

        self.hunger_level = max(0, self.hunger_level - amount)
        print(f"{self.name}가 사료를 {amount}만큼 먹었습니다.")

    def play(self, minutes):
        if minutes <= 0:
            print("놀이 시간은 1분 이상이어야 합니다.")
            return

        required_energy = minutes * 2

        if self.energy_level < required_energy:
            print("에너지가 부족합니다.")
            return

        self.energy_level -= required_energy
        self.hunger_level = min(100, self.hunger_level + minutes)
        print(f"{self.name}가 {minutes}분 동안 놀았습니다.")
```

```
    def sleep(self, hours):
        if hours <= 0:
            print("수면 시간은 1시간 이상이어야 합니다.")
            return

        self.energy_level = min(100, self.energy_level + hours * 20)
        print(f"{self.name}가 {hours}시간 동안 잠을 잤습니다.")

    def show_status(self):
        print("----- My Dog 상태 -----")
        print(f"이름: {self.name}")
        print(f"견종: {self.breed}")
        print(f"나이: {self.age}살")
        print(f"배고픔 정도: {self.hunger_level}")
        print(f"에너지 정도: {self.energy_level}")

my_dog = Dog("초코", "Maltese", 2)

my_dog.eat(30)
my_dog.play(20)
my_dog.sleep(2)
my_dog.show_status()
```

클래스

■ 클래스 사용해보기 2

CLASSUM

1. 카페 메뉴 클래스 정의해보기

- Class CafeMenu는 다음과 같은 속성을 받음
 - name
 - price
 - category
 - stock

```
name,price,category,stock  
아메리카노,2500,coffee,10  
카페라떼,4000,coffee,5  
치즈케이크,5500,dessert,2
```

2. 카페 메뉴 클래스 메서드 완성하기

- 할인적용: set_discount(rate) 메서드를 완성하여 가격(price)을 할인율(rate)만큼 인하하시오. Rate는 소수점 이하 한자리로 받으시오 (예: 0.1, 0.2...).
- 품절확인: is_soldout() 메서드를 완성하여 재고(stock)가 0이면 True를 반환하시오
- 판매 처리: sell(count) 메서드를 완성하여 판매 수량만큼 재고를 감소시키시오.
(단, 판매수량에 비해 재고량이 부족할 경우 "재고가 부족합니다 " 라고 출력하시오)

클래스

■ 클래스 사용해보기 2

CLASSUM

3. 호출해보기

- menu.csv의 아이템들을 사용해 클래스 호출하고 메소드 사용해보기
- 아메리카노에 10% 할인적용
- 치즈케이크와 카페라떼를 각 2개 판매 처리하고
- 각각 품질확인하시오

결과 예시

```
PS F:\한림대\데이터마이닝\samples> python .\cafe.py
아메리카노의 가격 할인: 2500원 -> 2250.0원
아메리카노의 가격: 2250.0원
카페라떼의 재고: 5
Alice가 카페라떼 2잔을 주문했습니다.
카페라떼의 재고: 3
치즈케이크의 재고: 2
Bob이 치즈케이크 2조각을 주문했습니다.
치즈케이크의 품질 여부: True
```

■ 클래스 사용해보기2

```
cafe.py > ...
1 class Menu:
2     def __init__(self, name, price, category, stock):
3         self.name = name
4         self.price = price
5         self.category = category
6         self.stock = stock
7
8     def set_discount(self, rate):
9         self.price = self.price * (1 - rate)
10
11    def is_soldout(self):
12        if self.stock == 0:
13            return True
14        return False
15
16    def sell(self, count):
17        if self.stock < count:
18            print("재고가 부족합니다.")
19            return
20        self.stock = self.stock - count
21
22
23 menu1 = Menu("아메리카노", 2500, "coffee", 10)
24 menu1.set_discount(0.1)
25 print(f"{menu1.name}의 가격: {menu1.price}원")
26
27 menu2 = Menu("카페라떼", 4000, "coffee", 5)
28 print(f"{menu2.name}의 재고: {menu2.stock}")
29 menu2.sell(2)
30 print(f"{menu2.name}의 재고: {menu2.stock}")
31
32 menu3 = Menu("치즈케이크", 5500, "dessert", 2)
33 print(f"{menu3.name}의 재고: {menu3.stock}")
34 menu3.sell(2)
35 print(f"{menu3.name}의 품절 여부: {menu3.is_soldout()}")
```

CLASSUM

정답

■ 심화

1. 기존 CafeMenu 클래스 새로운 속성 추가

- sold_count
 - 해당 메뉴의 누적 판매량
 - 초기값 0
- sales
 - 해당 메뉴의 누적 매출
 - 초기값 0

2. sell(count) 메서드 확장

- 판매 수량이 0 이하인 경우
 - "판매 수량은 1개 이상이어야 합니다." 출력
 - False 반환
- 판매 수량보다 재고가 적은 경우
 - "재고가 부족합니다." 출력
 - False 반환
- 판매에 성공한 경우
 - stock에서 판매 수량 차감
 - sold_count에 판매 수량 누적
 - sales에 판매 금액 누적
 - 판매 메뉴, 수량, 금액 출력
 - True 반환

■ 심화

3. restock(count) 메서드 추가

- 입고 수량이 0 이하인 경우
 - "입고 수량은 1개 이상이어야 합니다." 출력
- 정상적인 수량인 경우
 - stock에 입고 수량 추가
 - 메뉴 이름과 입고 수량 출력

4. show_info() 메서드 추가

- 다음 메뉴 정보 출력
 - 메뉴 이름
 - 카테고리
 - 현재 가격
 - 현재 재고
 - 누적 판매량
 - 누적 매출
 - 품절 여부

■ 심화

6. 메뉴 객체 생성

- 아메리카노
 - 가격: 2500원
 - 카테고리: coffee
 - 재고: 10개
- 카페라떼
 - 가격: 4000원
 - 카테고리: coffee
 - 재고: 5개
- 치즈케이크
 - 가격: 5500원
 - 카테고리: dessert
 - 재고: 2개

7. 메서드 호출

- 1) 아메리카노 10% 할인 적용
- 2) 아메리카노 2개 판매
- 3) 카페라떼 2개 판매
- 4) 치즈케이크 2개 판매
- 5) 치즈케이크 품질 여부 확인
- 6) 품질된 치즈케이크 1개 추가 판매 시도
- 7) 치즈케이크 3개 재입고
- 8) 치즈케이크 1개 재판매
- 9) 각 메뉴의 show_info() 호출

■ 심화

8. My Cafe 전체 매출 계산

- 세 메뉴 객체를 menu_list에 저장
- 반복문 또는 sum() 함수 사용
- 각 메뉴의 sales 값 합산
- 전체 매출 출력

출력 예시

My Cafe 전체 매출: 29000원

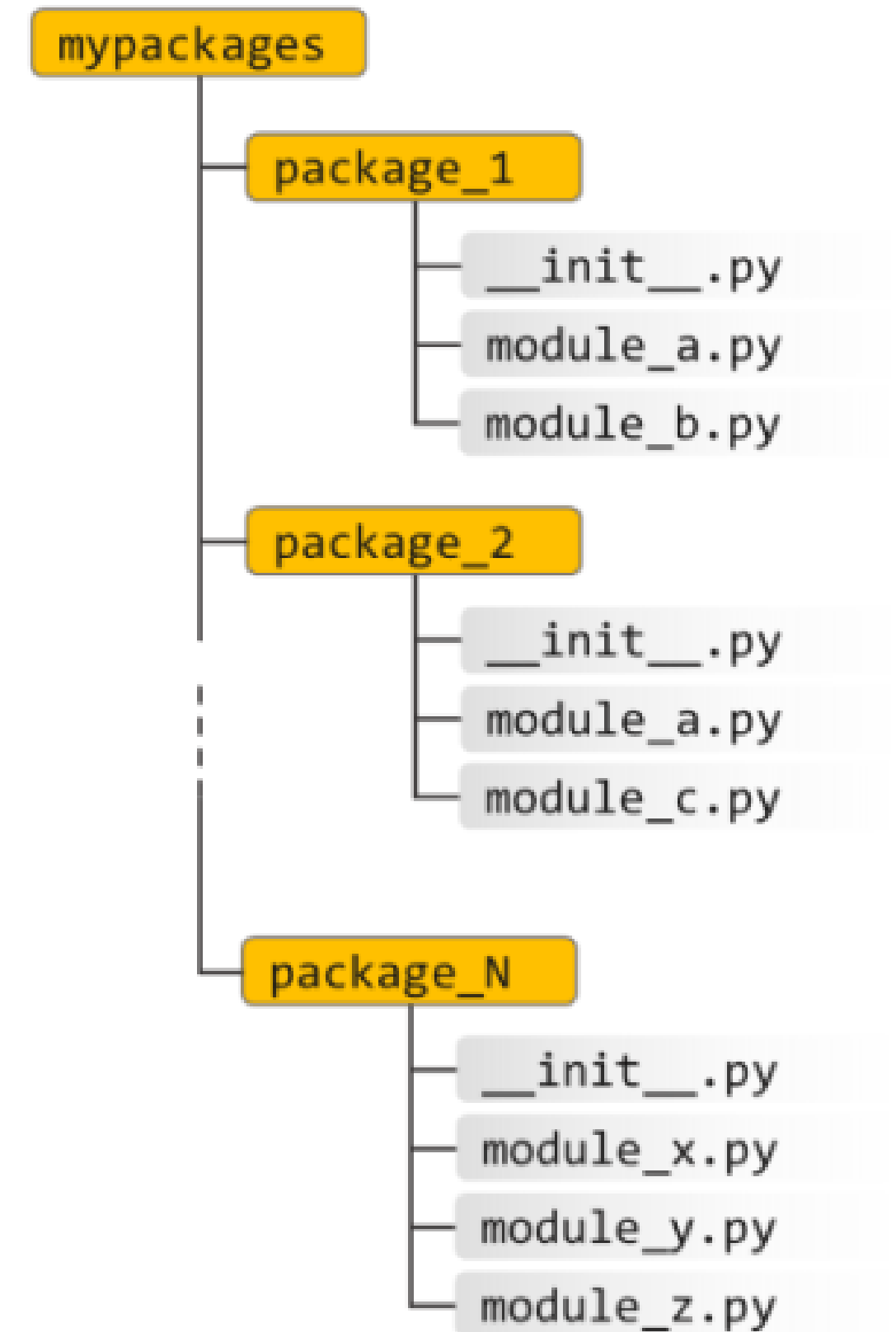
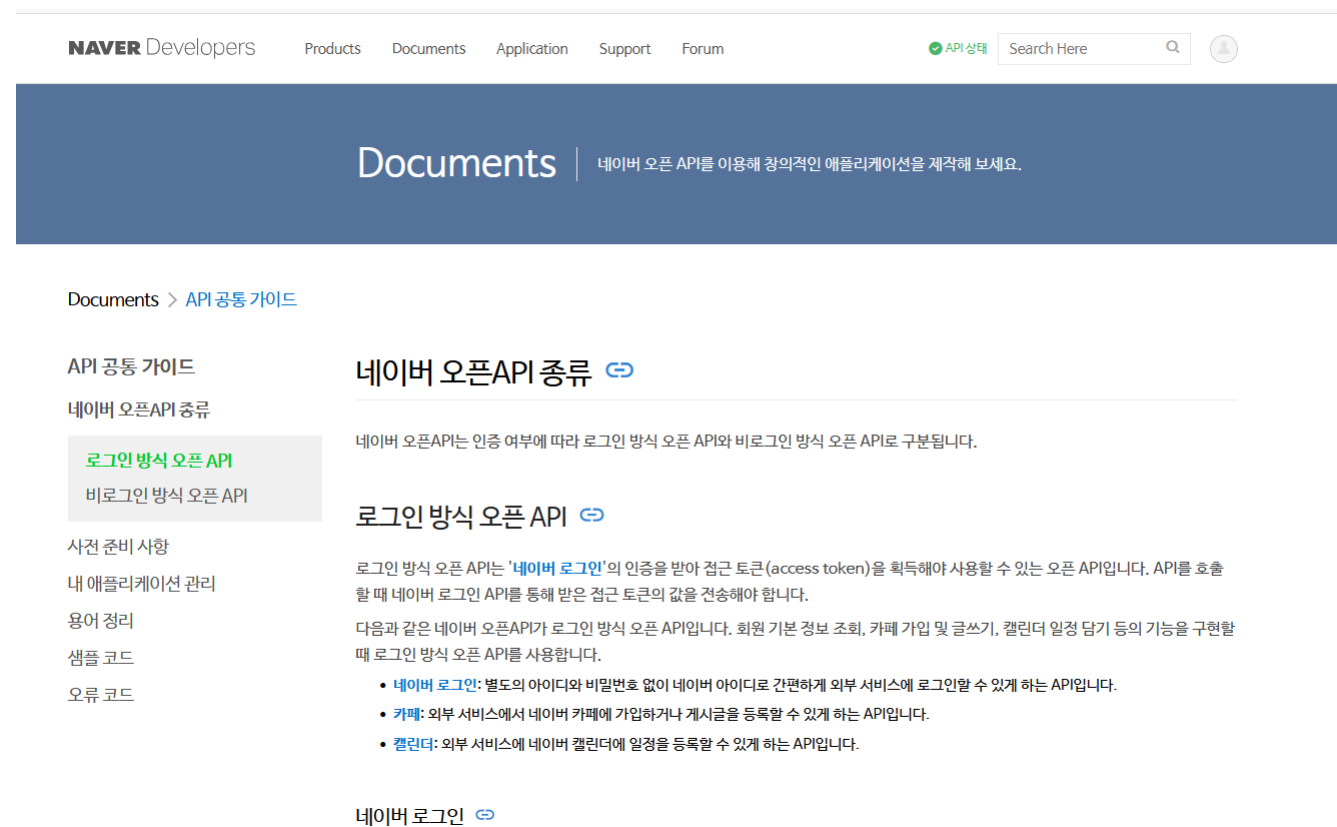
```
# 여러 메뉴 객체의 매출 합산
menu_list = [americano, cafe_latte, cheese_cake]
total_sales = sum(menu.sales for menu in menu_list)
print(f"\nMy Cafe 전체 매출: {total_sales}원")
```

모듬

안림대학

모듈

- 파이썬에서는 이미 작성된 프로그램을 모듈(module)
- 이 프로그램의 묶음을 패키지(packages)라고 함
- 모듈: 작은 프로그램 조각
 - 하나하나 연결해 어떤 목적을 가진 프로그램을 만들기 위한 작은 프로그램
- 인터페이스: 함수에서 매개변수를 입력하는 약속
 - 해당 모듈을 사용하기 위해서 모듈 간의 연결을 위한 약속



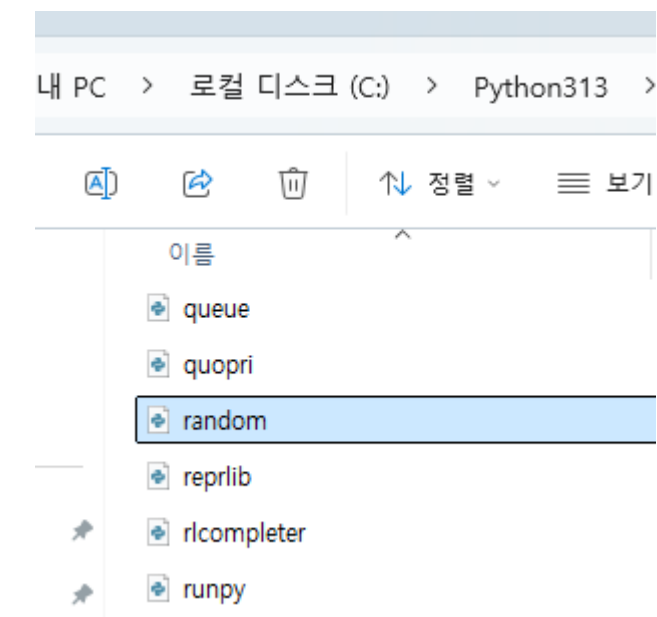
- 모듈의 예시: random, time

test.py

```
1 import random
2
3 print(random.randint(1, 100))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS F:\한림대\데이터마이닝\samples> python .\test.py
97
PS F:\한림대\데이터마이닝\samples> python .\test.py
46
PS F:\한림대\데이터마이닝\samples>
```



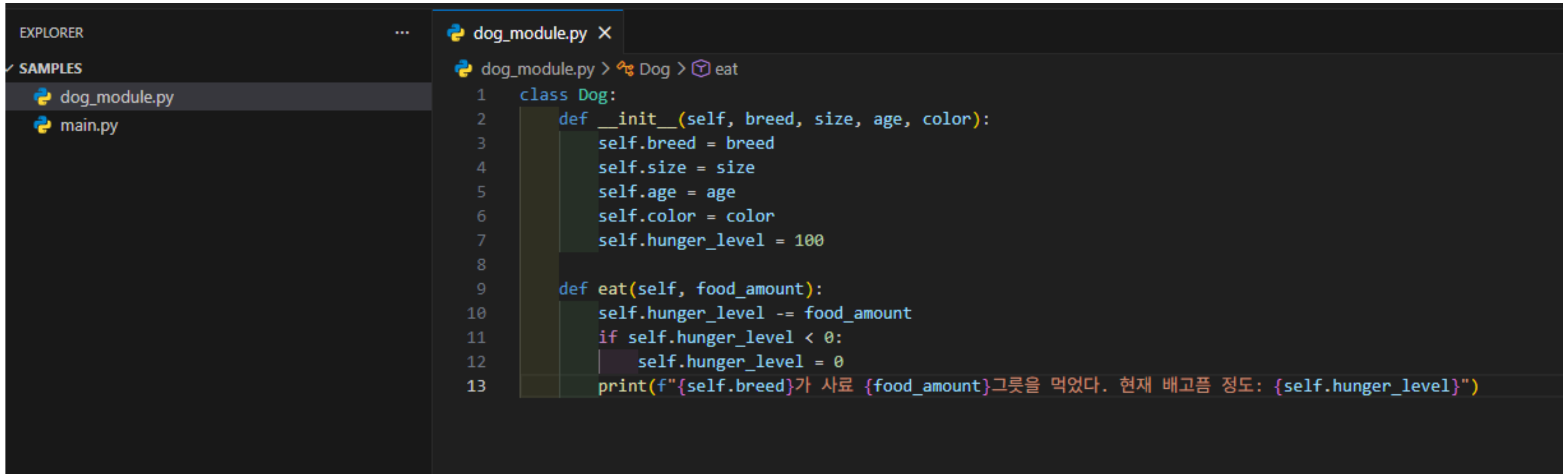
```
24 import time
25
26 for i in range(5):
27     print(f"{i+1}초")
28     time.sleep(1)
29
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS F:\한림대\데이터마이닝\samples> python .\main.py
1초
2초
3초
4초
```

모듈

- 모듈 만들어보기
 - 파이썬에서는 .py 파일 자체가 모듈

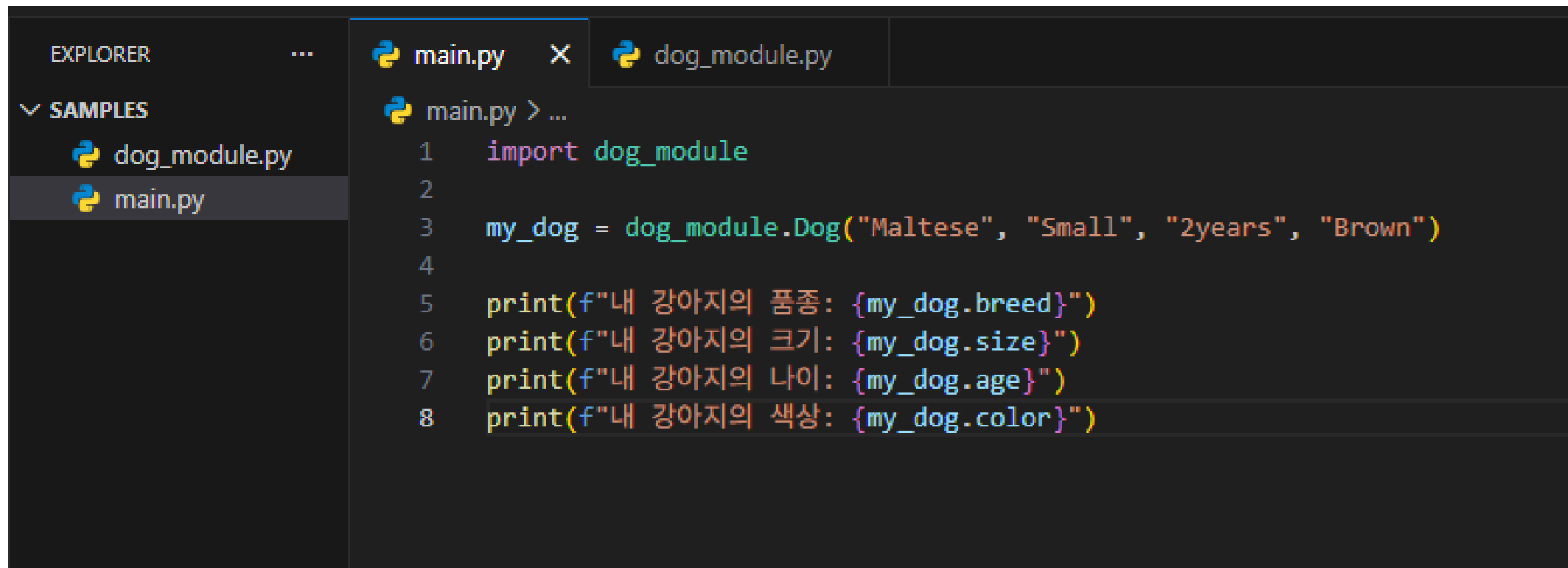


The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar displays a file tree under 'SAMPLES' containing 'dog_module.py' and 'main.py'. The main editor window shows the code for 'dog_module.py'. The code defines a 'Dog' class with an '__init__' method that initializes attributes: 'breed', 'size', 'age', 'color', and 'hunger_level' (set to 100). It also defines an 'eat' method that decreases 'hunger_level' by 'food_amount', ensuring it doesn't go below 0, and prints a message. The breadcrumb at the top of the editor indicates the path 'dog_module.py > Dog > eat'.

```
dog_module.py X
dog_module.py > Dog > eat
1 class Dog:
2     def __init__(self, breed, size, age, color):
3         self.breed = breed
4         self.size = size
5         self.age = age
6         self.color = color
7         self.hunger_level = 100
8
9     def eat(self, food_amount):
10        self.hunger_level -= food_amount
11        if self.hunger_level < 0:
12            self.hunger_level = 0
13        print(f"{self.breed}가 사료 {food_amount}그릇을 먹었다. 현재 배고픔 정도: {self.hunger_level}")
```

모듈

- 모듈 만들어보기
 - 파이썬에서는 .py 파일 자체가 모듈



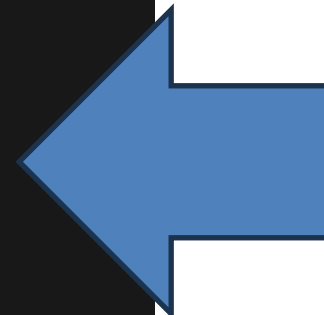
The screenshot shows a Python IDE with two files open: `main.py` and `dog_module.py`. The `EXPLORER` pane on the left shows the `SAMPLES` folder containing `dog_module.py` and `main.py`. The `main.py` file is selected, and its content is displayed in the editor. The code in `main.py` imports the `dog_module` and creates a `my_dog` object using the `Dog` class from `dog_module`. It then prints out the breed, size, age, and color of `my_dog`.

```
1 import dog_module
2
3 my_dog = dog_module.Dog("Maltese", "Small", "2years", "Brown")
4
5 print(f"내 강아지의 품종: {my_dog.breed}")
6 print(f"내 강아지의 크기: {my_dog.size}")
7 print(f"내 강아지의 나이: {my_dog.age}")
8 print(f"내 강아지의 색상: {my_dog.color}")
```

모듈

- 모듈 만들어보기
 - 파이썬에서는 .py 파일 자체가 모듈

```
PS F:\한림대\데이터마이닝\samples> python .\main.py
내 강아지의 품종: Maltese
내 강아지의 크기: Small
내 강아지의 나이: 2years
내 강아지의 색상: Brown
PS F:\한림대\데이터마이닝\samples> python .\main.py
내 강아지의 품종: Maltese
내 강아지의 크기: Small
내 강아지의 나이: 2years
내 강아지의 색상: Brown
Maltese가 사료 30그릇을 먹었다. 현재 배고픔 정도: 70
내 강아지의 배고픔 정도: 70
PS F:\한림대\데이터마이닝\samples> █
```



모듈

- 모듈 만들어보기
 - 파이썬에서는 .py 파일 자체가 모듈

```
main.py > ...
1  import dog_module
2
3  my_dog = dog_module.Dog("Maltese", "Small", "2years", "Brown")
4
5  print(f"내 강아지의 품종: {my_dog.breed}")
6  print(f"내 강아지의 크기: {my_dog.size}")
7  print(f"내 강아지의 나이: {my_dog.age}")
8  print(f"내 강아지의 색상: {my_dog.color}")
9
10 my_dog.eat(30)
11
12 print(f"내 강아지의 배고픔 정도: {my_dog.hunger_level}")
```

- 네임스페이스의 사용
 - 하나의 .py 파일 안에는 클래스.함수.변수가 있으며 import를 사용하여 호출할 수 있음
 - 단, 클라이언트 프로그램의 함수 이름과 호출된 모듈의 함수 이름이 같은 경우가 있음
- ➔ 호출된 모듈의 사용 범위를 명확히 지정하기 위해 사용하는 개념이 바로 네임스페이스

모듈

- 네임스페이스의 사용

(1) 모듈 이름에 별칭(alias)을 생성하여 모듈 안으로 코드를 호출하는 방법

```
14 import dog_module as dog
15
16 my_dog = dog.Dog("Maltese", "Small", "2years", "Brown")
17
18 print(f"내 강아지의 품종: {my_dog.breed}")
19 print(f"내 강아지의 크기: {my_dog.size}")
20 print(f"내 강아지의 나이: {my_dog.age}")
21 print(f"내 강아지의 색상: {my_dog.color}")
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS F:\한림대\데이터마이닝\samples> python .\main.py
내 강아지의 품종: Maltese
내 강아지의 크기: Small
내 강아지의 나이: 2years
내 강아지의 색상: Brown
PS F:\한림대\데이터마이닝\samples> 
```

모듈

- 네임스페이스의 사용

(2) from 구문을 사용하여 모듈에서 특정 함수 또는 클래스만 호출하는 방법

```
14 from dog_module import Dog
15
16 my_dog = Dog("Maltese", "Small", "2years", "Brown")
17
18 print(f"내 강아지의 품종: {my_dog.breed}")
19 print(f"내 강아지의 크기: {my_dog.size}")
20 print(f"내 강아지의 나이: {my_dog.age}")
21 print(f"내 강아지의 색상: {my_dog.color}")
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS F:\한림대\데이터마이닝\samples> python .\main.py

내 강아지의 품종: Maltese

내 강아지의 크기: Small

내 강아지의 나이: 2years

내 강아지의 색상: Brown

PS F:\한림대\데이터마이닝\samples> █

모듈

- 네임스페이스의 사용

(3) 해당 모듈 안에 있는 모든 함수, 클래스, 변수를 가져오는 *를 사용하는 것

```
from dog_module import *

my_dog = Dog("Maltese", "Small", "2years", "Brown")

print(f"내 강아지의 품종: {my_dog.breed}")
print(f"내 강아지의 크기: {my_dog.size}")
print(f"내 강아지의 나이: {my_dog.age}")
print(f"내 강아지의 색상: {my_dog.color}")
```

로그 출력

한림대학교

로그 출력

- 로깅(logging): 로그를 관리하는 기법. 프로그램이 실행되는 동안 일어나는 정보를 기록으로 남김 (디버깅에 반드시 필요!!!)
- 간단한 확인용으로는 print() 가 충분할 수 있지만,
- 실제로 개발을 하며 로그 데이터 관리 및 보관을 하기 위해서는 별도의 도구 사용 필수
- 파이썬에서는 주로 logging이라는 이름의 모듈을 사용

로그 출력

- 로깅(logging)

```
logger.py
1  import logging
2
3  logging.debug("틀렸잖아!")
4  logging.info("확인해!")
5  logging.warning("조심해!")
6  logging.error("에러 났어!!!")
7  logging.critical ("망했다....")
8
```

로그 출력

■ 로깅(logging)

```
logger.py
1  import logging
2
3  logging.debug("틀렸잖아!")
4  logging.info("확인해!")
5  logging.warning("조심해!")
6  logging.error("에러 났어!!!")
7  logging.critical ("망했다....")
8
```

```
PS F:\한림대\데이터마이닝\samples> python .\logger.py
WARNING:root:조심해!
ERROR:root:에러 났어!!!
CRITICAL:root:망했다....
```

로그 출력

■ 로깅의 출력 5단계

- ① 1단계: DEBUG
 - 개발 시점에 프로그램이 문제없이 실행되는지 확인하기 위해 출력하는 결과임
- ② 2단계: INFO
 - 기본적으로 사용자에게 실행 결과를 알려주는 로그 정보
- ③ 3단계: WARNING
 - 문제가 될 수 있는 상황을 기록하는 것
- ④ 4, 5단계: ERROR, CRITICAL
 - 에러가 발생하거나 프로그램이 더이상 수행하기 어려울 때 발생하는 에러

Logger의 기본 레벨이 Warning으로 되어 그 이전 단계가 안보이는 것!!

로그 출력

- `basicConfig()` 설정하여 로깅 레벨 변경

```
9  import logging
10
11  logging.basicConfig(level=logging.INFO)
12
13  logger = logging.getLogger(__name__)
14
15  logger.warning("warning 보임")
16  logger.info("info 보임")
```

현재 파이썬 파일의 이름(모듈 이름)

```
PS F:\한림대\데이터마이닝\samples> python .\logger.py
WARNING: __main__:warning 보임
INFO: __main__:info 보임
```

로그 출력

▪ basicConfig() 설정하여 로깅 레벨 변경

다른 파일에서 import해서 실행 할 경우

```

main.py  X  logger.py  dog_module.py
main.py > ...
1  import dog_module
2  from logger import logger
3
4  my_dog = dog_module.Dog("Maltese", "Small", "2years", "Brown")
5
6  print(f"내 강아지의 품종: {my_dog.breed}")
7  print(f"내 강아지의 크기: {my_dog.size}")
8  print(f"내 강아지의 나이: {my_dog.age}")
9  print(f"내 강아지의 색상: {my_dog.color}")
10
11 my_dog.eat(30)
12
13 logger.info(f"내 강아지의 배고픔 정도: {my_dog.hunger_level}")
14
15 print(f"내 강아지의 배고픔 정도: {my_dog.hunger_level}")
16

```

```

PS F:\한림대\데이터마이닝\samples> python .\main.py
내 강아지의 품종: Maltese
내 강아지의 크기: Small
내 강아지의 나이: 2years
내 강아지의 색상: Brown
Maltese가 사료 30그릇을 먹었다. 현재 배고픔 정도: 70
INFO:logger:내 강아지의 배고픔 정도: 70
내 강아지의 배고픔 정도: 70

```

오늘은 여기까지.

Thank you for listening.